SJSU CMPE 172

CMPE 172 – MILESTONE 1 REPORT

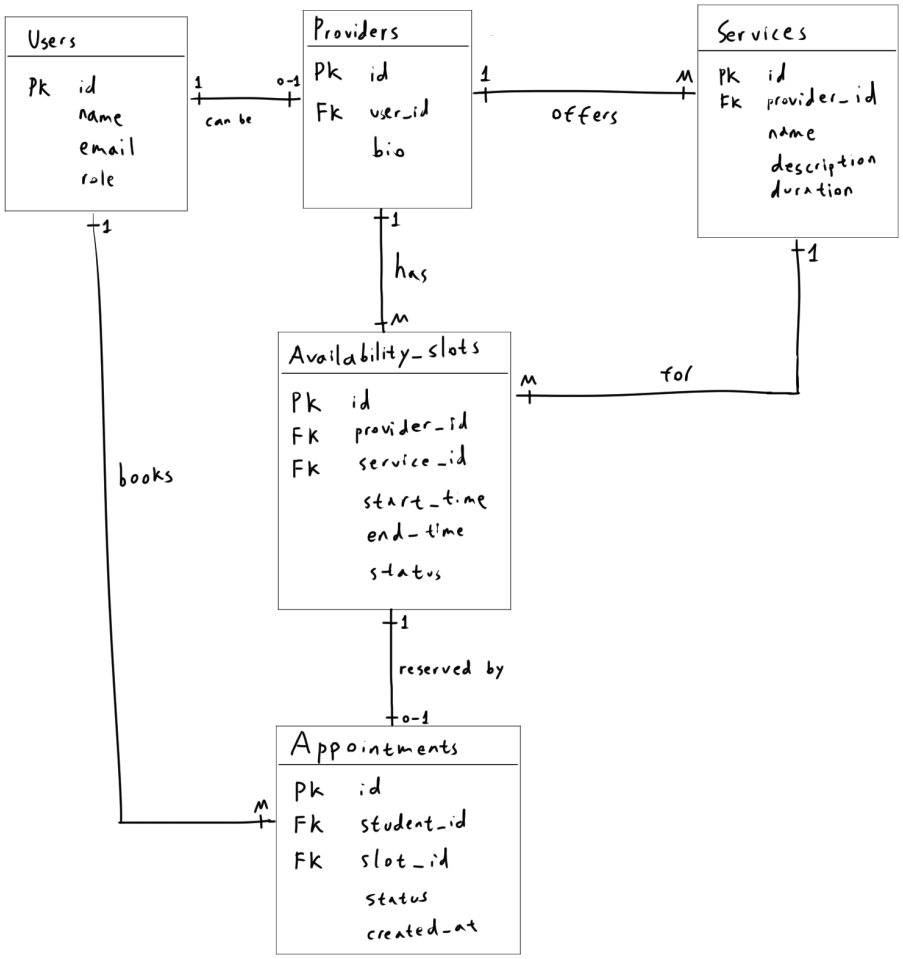
- **Name: Canhui Huang**
- **Student ID: 017553913**
- **Date: 09/24/2026**
- **Chosen scenario: Students can browse and book appointments with tutors**
- **GitHub repository (milestone tag):**
- **Code-walkthrough video link:**

1. Overview

This milestone implements the initial design and skeleton for backend for a tutoring appointment system. The application uses Spring Boot with a layered Controller, Service, and Repository architecture, JDBC for database access, and an H2 database for local development.

2. Design and Implementation

Database design



Main Entities

Users store user information for both student and tutor roles. Each user has an unique id, name, email, and role.

Tutors store tutor specific information such as bio in a separate table. A user can be a provider if the role is tutor.

Services store service information that a provider provides such as tutoring for Calculus, Math, etc. Relevant information are name, description, and duration of the service. A provider can offer many services.

Availability_slots store availability slots provided by a provider. This also makes reference to a service that the provider is providing. Information stored are: start_time, end_time, and status. A provider can offer many slots.

Appointments store appointments booked by users. An appointment has to refer to an availability_slot, and the information stored are: status and created_at.

Relational Schema

```SQL
CREATE TABLE users (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    role VARCHAR(20) NOT NULL
);

CREATE TABLE providers (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id BIGINT NOT NULL UNIQUE,
    bio VARCHAR(500),

    FOREIGN KEY (user_id) REFERENCES users(id)
);

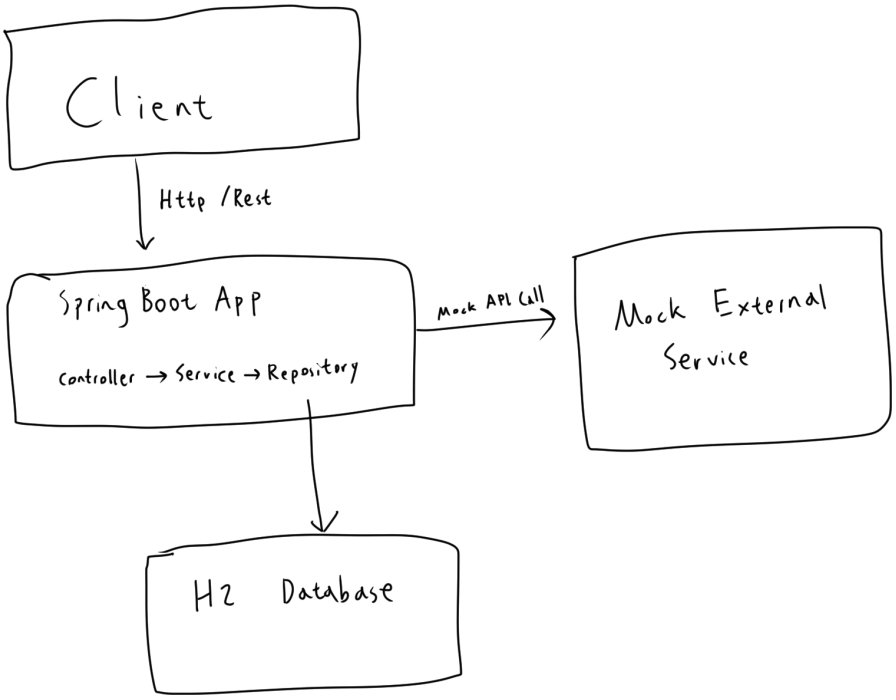
CREATE TABLE services (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    provider_id BIGINT NOT NULL,
    name VARCHAR(100) NOT NULL,
    description VARCHAR(500),
    duration INT NOT NULL,
    FOREIGN KEY (provider_id) REFERENCES providers(id)
);

CREATE TABLE availability_slots (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    provider_id BIGINT NOT NULL,
    service_id BIGINT NOT NULL,
    start_time TIMESTAMP NOT NULL,
    end_time TIMESTAMP NOT NULL,
    status VARCHAR(20) NOT NULL,

    FOREIGN KEY (provider_id) REFERENCES providers(id),
    FOREIGN KEY (service_id) REFERENCES services(id)
);

CREATE TABLE appointments (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    student_id BIGINT NOT NULL,
    slot_id BIGINT NOT NULL UNIQUE,
    status VARCHAR(20) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES users(id),
    FOREIGN KEY (slot_id) REFERENCES availability_slots(id)
);
```

Block Diagram



Request Flow through the Layers

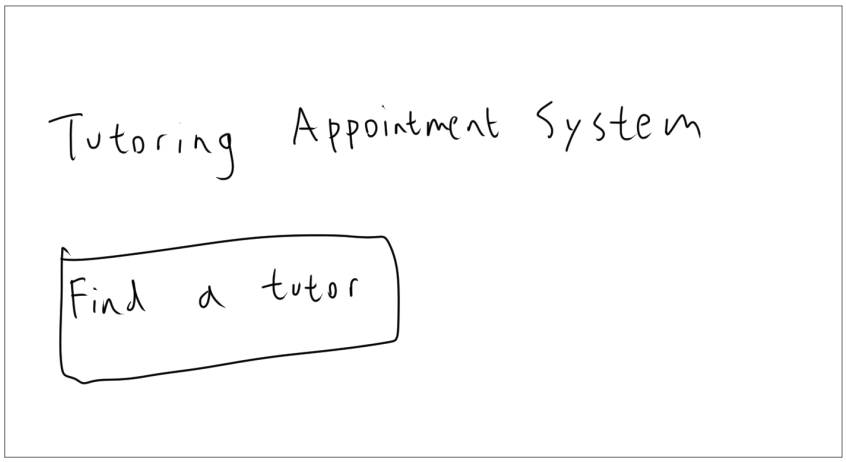
GET /slots → SlotController → SlotService → SlotRepository → JdbcTemplate → H2 Database → SlotDto → JSON Response

Page Controller vs Front Controller

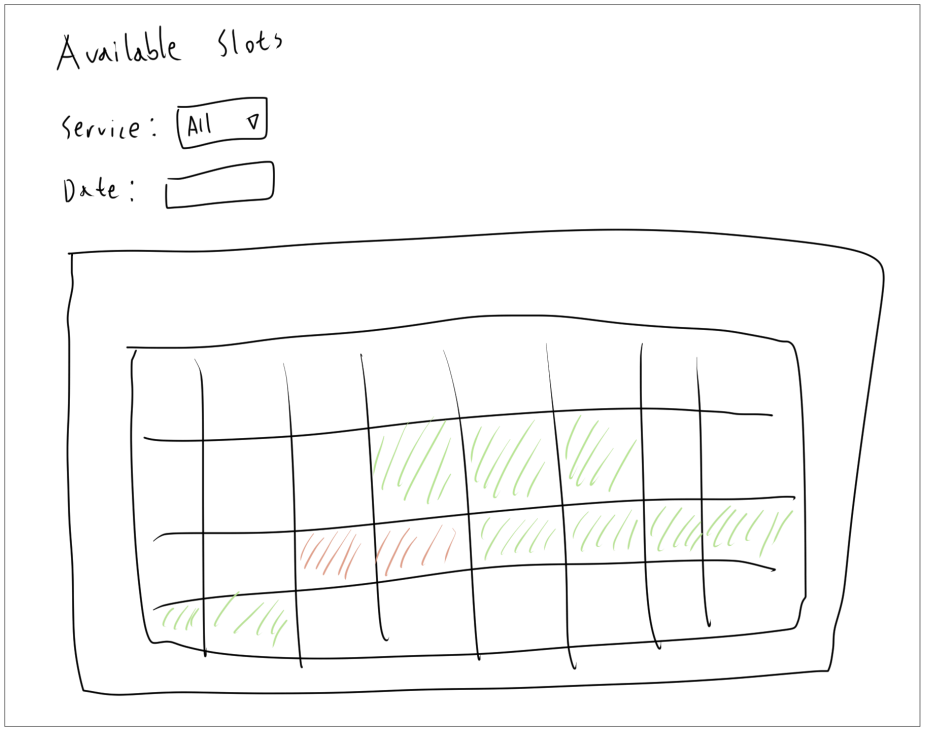
A Page Controller approach uses a separate controller to handle requests for individual pages or actions. In contrast, the Front Controller pattern uses a centralized controller as the entry point for incoming requests and then routes them to the appropriate application controller. This project follows the Front Controller pattern because Spring Boot uses Spring MVC's DispatcherServlet to receive HTTP requests and route them to the appropriate controller method. For example, a GET /slots request is first handled by the Spring MVC infrastructure and then routed to SlotController.

Wireframes

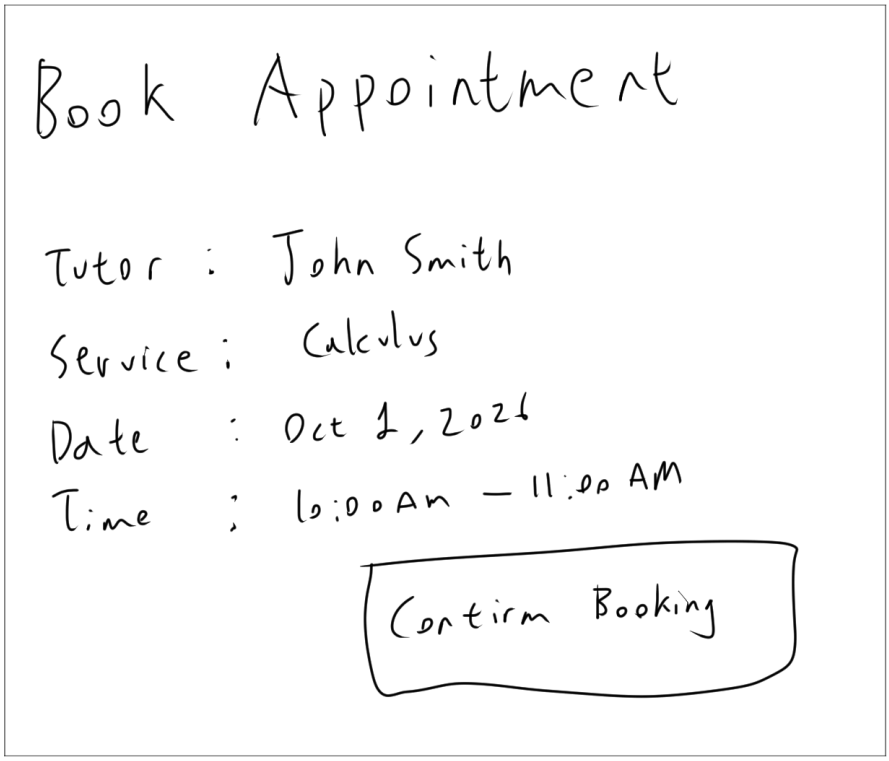
Home



View Available Slots



Book Appointment



Confirmation

